

## Ten thousand what-ifs

### Follow-along worksheet

Your goal: compare possible profits and explain the trade-off.  
Then connect an ordinary Excel input to a Python chart.

Name

An uncertain input I plan around

Open follow-along.xlsx. Add a blank simulation sheet and move it first.  
Keep the other sheets. Windows: select the cell > Formulas > Insert Python.  
Ctrl+Enter runs it. Use Excel Value to see numbers in the grid.

### The assumptions

Buy 1,100 units at \$30 each. Sell each for \$50. Unsold stock has no resale value. Demand is normal, centered at 1,000, with standard deviation 200, bounded at zero. Other costs are excluded. This simplified example treats demand as continuous, so simulated units can include fractions.

### 1 Simulate 10,000 possible profits

TRY IT / simulation!A3. The seed makes our example repeatable.

```
np.random.seed(42)
demand = np.maximum(np.random.normal(1000, 200, 10000), 0)
units_sold = np.minimum(demand, 1100)
profit = units_sold * 50 - 1100 * 30
round(profit.mean())
```

Average simulated profit (rounded dollars)

Check: about \$14,998. We pay for all stock, even when demand is low.

# The downside matters too

Work on simulation. Run the steps in order so profit is available.

## 2 See the range of profits

TRY IT / A6. Solid line = \$0. Dashed line = average profit.

```
plt.figure()
sns.histplot(profit, bins=range(-25000, 25001, 5000),
             color="#CF3338")
plt.axvline(0, color="black")
plt.axvline(profit.mean(), color="#707070", linestyle="--")
```

Where do the money-losing outcomes appear?

## 3 Calculate the loss rate

TRY IT / A10. True/False comparisons can be averaged.

```
(profit < 0).mean()
```

Loss rate as a percentage

## 4 Look at lower, middle and higher outcomes

WATCH / A14. P10 means roughly the lowest 10% end here.

```
pd.Series(np.percentile(profit, [10, 50, 90]).round(),
         index=["P10", "P50", "P90"])
```

Checks: losses are below \$0; loss rate 4.65%. P10 \$4,077; P50 \$16,974; P90 \$22,000. P50 is the median, not the mean. Many outcomes tie at the \$22,000 stock-limited maximum. These are simulated results, not guarantees.

Which number adds most to your understanding beyond the average?

# Compare the stock choices

The same demand draws let us change just the stock quantity.

## 5 Test 900 units against 1,100

WATCH / simulation!A20. Run after the earlier simulation cells.

```
profit_900 = np.minimum(demand, 900) * 50 - 900 * 30
pd.DataFrame({"Stock": [900, 1100],
              "Mean profit": [profit_900.mean(), profit.mean()],
              "Loss rate": [(profit_900 < 0).mean(), (profit < 0).mean()]})
```

| Stock       | Average profit | Loss rate |
|-------------|----------------|-----------|
| 900 units   | \$16,007       | 1.06%     |
| 1,100 units | \$14,998       | 4.65%     |

Under our assumptions, 900 has the higher average and fewer losses. Its maximum profit is lower: \$18,000 versus \$22,000. These two choices do not tell us the best possible stock level for every situation.

**Which stock quantity would you choose, and why?**

**Which assumption would you check before applying this at work?**

Possible next checks: resale value of leftovers, a different demand pattern, service targets or the cost of lost sales.

# An Excel input for your chart

Work on moving-average. There is one row per day in this workbook.

## 6 Smooth daily sales

TRY IT / G1. A seven-day window uses this day and the previous six.

```
sales_df = xl("daily_sales[#All]", headers=True)
sales_df["Date"] = pd.to_datetime(sales_df["Date"])
sales_df = sales_df.sort_values("Date")
sales_df["Average"] = sales_df["Sales"].rolling(7).mean()
sales_df.plot(x="Date", y=["Sales", "Average"],
              color=["#707070", "#CF3338"])
```

The first six averages are blank until seven days are available.  
Smoothing describes recent history. It does not predict tomorrow.

## 7 Control the window from Excel

TRY IT / E1 is an ordinary Excel cell. Enter 7. Add Data Validation:  
Allow Whole number, between 1 and 60. Use the Stop error alert.  
In G20, insert Python and run the code below. Then change E1 to 30.

```
moving_n = int(xl("'moving-average'!E1"))
sales_df["Average"] = sales_df["Sales"].rolling(moving_n).mean()
sales_df.plot(x="Date", y=["Sales", "Average"],
              color=["#707070", "#CF3338"])
plt.title(f"{moving_n}-day average")
```

**What happens when you change the window from 7 to 30?**

Check: the line gets smoother and reacts more slowly. Validation helps with typed entries, but pasting can bypass it. Keep E1 a whole number from 1 to 60. If your real data has missing days, seven rows may not be a week.

**An input I would let someone change in my own workbook**

# Your next step

Use one small example to answer a question from your own work.

**A question I want to answer in my own workbook**

**A familiar result or simple baseline I will check against**

**When I will practice**

**One question I still have**

## If you get stuck

Use this worksheet's cell locations and run steps in order.

#PYTHON!: check names, quotes and whether earlier cells ran.

#SPILL!: clear space for the result. #BUSY!: wait for calculation.

An Image result is normal. Right-click it > Display Plot over Cells.

## Keep building with the book

### Python in Excel for Data Analytics

George Mount / Packt

Chapters 8 and 10: simulation and Excel integration.

Build examples with clear assumptions  
and inputs your colleagues can use.



### Preorder your copy

[stringfestanalytics.com/pyxda](https://stringfestanalytics.com/pyxda)

Free files and replay: [stringfestdata.gumroad.com/l/pxlaf](https://stringfestdata.gumroad.com/l/pxlaf)

Help: Microsoft Python in Excel getting started guide